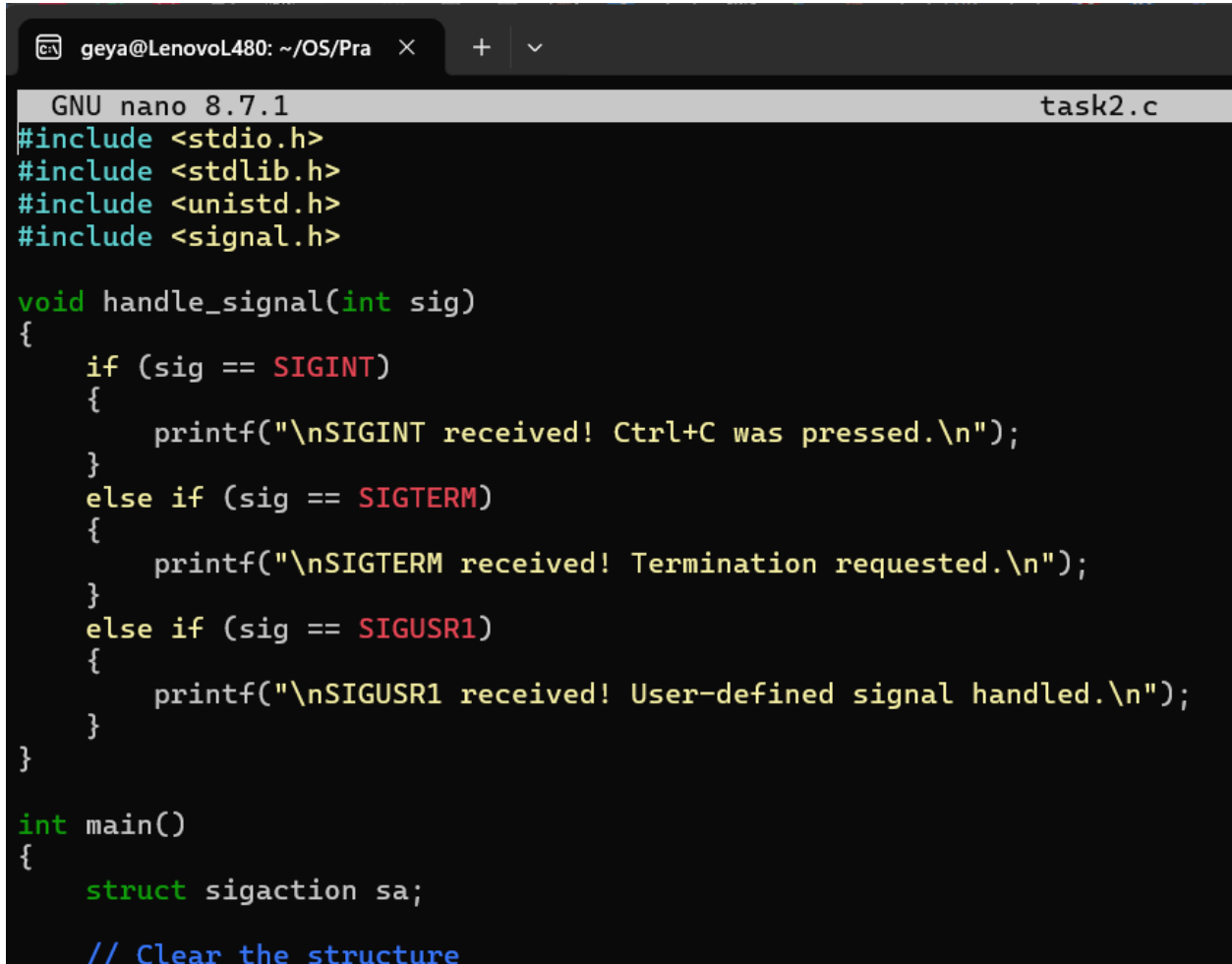


Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Code:



```
geya@LenovoL480: ~/OS/Pra × + v
GNU nano 8.7.1 task2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received! Ctrl+C was pressed.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received! Termination requested.\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received! User-defined signal handled.\n");
    }
}

int main()
{
    struct sigaction sa;

    // Clear the structure
```

```

// Clear the structure
sa.sa_handler = handle_signal;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

// Register signal handlers
sigaction(SIGINT, &sa, NULL);
sigaction(SIGTERM, &sa, NULL);
sigaction(SIGUSR1, &sa, NULL);

printf("Signal handling program started.\n");
printf("Process ID (PID): %d\n", getpid());
printf("Press Ctrl+C to send SIGINT.\n");
printf("Use 'kill -SIGTERM %d' to send SIGTERM.\n", getpid());
printf("Use 'kill -SIGUSR1 %d' to send SIGUSR1.\n", getpid());

while (1)
{
    printf("Program is running...\n");
    sleep(3);
}

return 0;
}

```

Output:

<pre> priyanka@DESKTOP-H0IS07A:~\$ cd ~/OS_Skill/Week-4 priyanka@DESKTOP-H0IS07A:~/OS_Skill/Week-4\$ gcc src/signal_handler.c -o bin/signal_handler priyanka@DESKTOP-H0IS07A:~/OS_Skill/Week-4\$./bin/signal_handler Program running... PID: 890 Send signals using kill command. ^C SIGINT received! SIGUSR1 received! ^Z [1]+ Stopped ./bin/signal_handler priyanka@DESKTOP-H0IS07A:~/OS_Skill/Week-4\$ </pre>	<pre> priyanka@DESKTOP-H0IS07A:~\$ cd ~/OS_Skill/Week-4 priyanka@DESKTOP-H0IS07A:~/OS_Skill/Week-4\$ kill -SIGUSR1 890 priyanka@DESKTOP-H0IS07A:~/OS_Skill/Week-4\$ </pre>
---	--